

AKADEMIA TARNOWSKA

Katedra Informatyki

KONSPEKT LABORATORIUM 9

Integracja z REST API

Zaawansowany interfejs użytkownika

Kierunek: Informatyka (II stopień) | Rok 1, Semestr 1 | Studia stacjonarne

Prowadzący:	mgr inż. Dominik Aksamił
Czas zajęć:	90 minut (2 x 45 min)
Efekty kształcenia:	IN2_U04, IN2_U02, IN2_U03

1. Cele zajęć i efekty kształcenia

Zajęcia laboratoryjne §9 są poświęcone praktycznej integracji interfejsu użytkownika z zewnętrznymi źródłami danych za pośrednictwem REST API. Student nabywa kompetencje wymagane przez efekt IN2_U04, budując od podstaw w pełni funkcjonalną aplikację webową.

1.1. Powiązanie z efektami kształcenia

Kod efektu	Treść efektu	Sposób weryfikacji
IN2_U04	Posiada umiejętność integracji interfejsów użytkownika z aplikacjami serwerowymi, projektowania i obsługi interakcji z wykorzystaniem API	Obserwacja zachowań, kod oddany do oceny
IN2_U02	Potrafi analizować i interpretować dane dotyczące zachowań użytkowników, iteracyjnie doskonaląc interfejsy	Aktywność na zajęciach
IN2_U03	Potrafi projektować i implementować UI zgodnie z zasadami UCD, dostosowując rozwiązania do potrzeb użytkownika	Kolokwium, obserwacja zachowań
IN2_W01	Posiada zaawansowaną wiedzę dotyczącą wykorzystania bibliotek webowych (React.js)	Kolokwium

1.2. Cele szczegółowe — po zajęciach student:

- konfiguruje React Query v5 (QueryClient, QueryClientProvider, queryKeys) w projekcie Vite + React
- implementuje własne hooki (custom hooks) opakowujące zapytania do REST API
- obsługuje poprawnie wszystkie stany cyklu życia zapytania: ładowanie, błąd, pusty wynik, sukces
- stosuje debouncing w polu wyszukiwania, aby ograniczyć liczbę zbędnych żądań HTTP
- implementuje paginację klasyczną oraz opcjonalnie nieskończone przewijanie (infinite scroll)
- persystuje dane lokalnie (ulubione filmy) za pomocą localStorage
- konfiguruje mockowanie HTTP przy użyciu MSW.js lub rozszerzenia Requestly
- zabezpiecza klucz API poprzez zmienne środowiskowe (.env) w Vite

2. Plan zajęć — harmonogram 90 minut

Czas	Blok tematyczny	Forma	Efekt
0–10 min	Wprowadzenie: REST API, React Query v5 — motywacja i architektura	Mini-wykład	IN2_W01
10–20 min	Konfiguracja projektu: Vite, instalacja pakietów, .env, QueryClientProvider	Praktyczna	IN2_U04
20–35 min	Warm-up z Rick & Morty API — custom hook useCharacters, obsługa stanów	Praktyczna	IN2_U04
35–55 min	Movie Browser: hook useFetchMovies, MovieCard z optimistic update ulubionych	Praktyczna	IN2_U04
55–65 min	Wyszukiwanie z debouncingiem (useDebounce), paginacja klasyczna	Praktyczna	IN2_U02
65–75 min	Modal szczegółów filmu, persystencja ulubionych w localStorage	Praktyczna	IN2_U03
75–85 min	Rozszerzenie: infinite scroll z useInfiniteQuery + IntersectionObserver	Praktyczna	IN2_U04
85–90 min	Podsumowanie, kryteria oceny, pytania	Dyskusja	—

3. Zadanie: aplikacja Movie Browser

Celem laboratorium jest zbudowanie aplikacji Movie Browser integrującej się z The Movie Database (TMDB) API. Jako punkt startowy (warm-up, bez konieczności rejestracji klucza) wykorzystujemy publiczne Rick and Morty API.

Rick & Morty API (<https://rickandmortyapi.com>) jest w pełni publiczne — brak klucza API. Idealne do demonstracji i debugowania bez konfiguracji konta.

3.1. Zakres funkcjonalny aplikacji

#	Funkcja	API endpoint	Priorytet
1	Lista popularnych filmów z paginacją	GET /movie/popular?page={n}	Obowiązkowy
2	Wyszukiwanie filmów z debouncingiem (300 ms)	GET /search/movie?query={q}	Obowiązkowy
3	Szczegóły filmu (modal lub osobna strona /movie/:id)	GET /movie/{id}	Obowiązkowy
4	Ulubione — dodawanie/usuwanie, persystencja w localStorage	— (lokalne)	Obowiązkowy
5	Infinite scroll zastępujący paginację klasyczną	GET /movie/popular?page={n}	Rozszerzenie
6	Filtrowanie po gatunku (genre filter)	GET /genre/movie/list	Rozszerzenie

3.2. Struktura katalogów projektu

Zalecana struktura projektu:

```
src/  
  api/  
    tmdbClient.ts      // axios instance + interceptory  
    endpoints.ts       // stałe URL  
  hooks/  
    useFetchMovies.ts  // React Query — lista filmów  
    useMovieDetails.ts // React Query — szczegóły  
    useDebounce.ts     // debounce hook (300 ms)  
    useFavorites.ts    // localStorage + optimistic UI  
    useCharacters.ts    // warm-up: Rick & Morty API  
  components/  
    MovieCard.tsx      // karta z optimistic update  
    MovieModal.tsx     // modal ze szczegółami  
    SkeletonCard.tsx   // skeleton loading  
    ErrorBanner.tsx    // error state  
    EmptyState.tsx     // empty state  
  mocks/  
    handlers.ts        // MSW — definicje handlerów  
    browser.ts         // MSW — worker setup  
  constants/  
    queryKeys.ts       // klucze React Query (enum)  
  App.tsx  
  main.tsx  
  .env                 // VITE_TMDB_API_KEY=...
```

4. Konfiguracja projektu — krok po kroku

4.1. Inicjalizacja projektu Vite + React + TypeScript

```
# Utwórz nowy projekt
npm create vite@latest movie-browser -- --template react-ts
cd movie-browser

# Instalacja React Query v5 i narzędzi pomocniczych
npm install @tanstack/react-query @tanstack/react-query-devtools
npm install axios
npm install -D msw

# Inicjalizacja MSW (Mock Service Worker)
npx msw init public/ --save
```

4.2. Zmienne środowiskowe — klucz API TMDB

Klucz API nigdy nie powinien znaleźć się w repozytorium. Vite udostępnia zmienne poprzez prefix VITE_.

```
# Plik: .env (dodaj do .gitignore!)
VITE_TMDB_API_KEY=twój_klucz_api_tutaj
VITE_TMDB_BASE_URL=https://api.themoviedb.org/3

# Plik: .env.example (wersjonuj w git — bez wartości)
VITE_TMDB_API_KEY=
VITE_TMDB_BASE_URL=https://api.themoviedb.org/3
```

```
// src/api/tmdbClient.ts
import axios from 'axios';

export const tmdbClient = axios.create({
  baseURL: import.meta.env.VITE_TMDB_BASE_URL,
  params: {
    api_key: import.meta.env.VITE_TMDB_API_KEY,
    language: 'pl-PL',
  },
});
```

4.3. QueryClient i QueryClientProvider

React Query wymaga jednego globalnego QueryClient opakowanego w QueryClientProvider na poziomie korzenia aplikacji.

```
// src/main.tsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 5, // 5 minut — dane 'świeże'
      retry: 2, // 2 ponowne próby przy błędzie
      refetchOnWindowFocus: false,
    },
  },
});

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
```

```
    <App />
    <ReactQueryDevtools initialIsOpen={false} />
  </QueryClientProvider>
</StrictMode>
);
```

4.4. Klucze zapytań (queryKeys) — dobre praktyki

Klucze zapytań powinny być scentralizowane jako stałe. Zapobiega to literówkom i ułatwia inwalidację cache.

```
// src/constants/queryKeys.ts
export const QUERY_KEYS = {
  movies: {
    all: () => ['movies'] as const,
    popular: (page: number) => ['movies', 'popular', page] as const,
    search: (query: string, page: number) => ['movies', 'search', query, page] as
const,
    detail: (id: number) => ['movies', 'detail', id] as const,
  },
  genres: {
    all: () => ['genres'] as const,
  },
  // Rick & Morty (warm-up)
  characters: {
    all: () => ['characters'] as const,
    page: (page: number) => ['characters', page] as const,
  },
} as const;
```

5. Warm-up: Rick & Morty API

Przed pracą z TMDB (wymagający klucz API) studenci ćwiczą wzorzec zapytań na publicznym Rick and Morty API. API jest identyczne w strukturze (paginacja, wyszukiwanie), lecz nie wymaga żadnej autoryzacji.

<https://rickandmortyapi.com/api/character?page=1&name=rick> — brak klucza, pełna paginacja, idealne do nauki!

5.1. Hook useCharacters

```
// src/hooks/useCharacters.ts
import { useQuery } from '@tanstack/react-query';
import axios from 'axios';
import { QUERY_KEYS } from '../constants/queryKeys';

const RAM_BASE = 'https://rickandmortyapi.com/api';

interface Character {
  id: number;
  name: string;
  status: 'Alive' | 'Dead' | 'unknown';
  species: string;
  image: string;
}

interface RamResponse {
  info: { count: number; pages: number; next: string | null };
  results: Character[];
}

export function useCharacters(page = 1, name = '') {
  return useQuery({
    queryKey: QUERY_KEYS.characters.page(page),
    queryFn: async () => {
      const params = new URLSearchParams({ page: String(page) });
      if (name) params.set('name', name);
      const { data } = await axios.get<RamResponse>(
        `${RAM_BASE}/character?${params}`
      );
      return data;
    },
    placeholderData: (prev) => prev, // zachowuje poprzednie dane przy zmianie strony
  });
}
```

5.2. Ćwiczenie warm-up (5 min)

1. Użyj hooka useCharacters w komponencie App.tsx
2. Wyświetl imię i zdjęcie pierwszych 20 postaci
3. Dodaj przyciski Poprzednia / Następna strona
4. Zaobserwuj w React Query DevTools, jak zmienia się stan cache

6. Główne hooki React Query — TMDB

6.1. Hook useFetchMovies — lista filmów z paginacją

```
// src/hooks/useFetchMovies.ts
import { useQuery } from '@tanstack/react-query';
import { tmdbClient } from '../api/tmdbClient';
import { QUERY_KEYS } from '../constants/queryKeys';

export interface Movie {
  id: number;
  title: string;
  overview: string;
  poster_path: string | null;
  release_date: string;
  vote_average: number;
  genre_ids: number[];
}

interface MoviesResponse {
  page: number;
  results: Movie[];
  total_pages: number;
  total_results: number;
}

export function useFetchMovies(page = 1, query = '') {
  const isSearch = query.trim().length > 0;

  return useQuery({
    queryKey: isSearch
      ? QUERY_KEYS.movies.search(query, page)
      : QUERY_KEYS.movies.popular(page),
    queryFn: async () => {
      const endpoint = isSearch ? '/search/movie' : '/movie/popular';
      const params: Record<string, string | number> = { page };
      if (isSearch) params.query = query;
      const { data } = await tmdbClient.get<MoviesResponse>(endpoint, { params });
      return data;
    },
    enabled: !isSearch || query.trim().length >= 2, // min 2 znaki dla wyszukiwania
    placeholderData: (prev) => prev, // płynna paginacja bez migotania
    staleTime: 1000 * 60 * 3, // 3 minuty
  });
}
```

6.2. Hook useDebounce — ograniczenie liczby zapytań

```
// src/hooks/useDebounce.ts
import { useState, useEffect } from 'react';

export function useDebounce<T>(value: T, delay = 300): T {
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}

// Użycie w komponencie:
// const [query, setQuery] = useState('');
// const debouncedQuery = useDebounce(query, 300);
// const { data } = useFetchMovies(page, debouncedQuery);
```


6.3. Hook useMovieDetails — szczegóły filmu

```
// src/hooks/useMovieDetails.ts
import { useQuery } from '@tanstack/react-query';
import { tmdbClient } from '../api/tmdbClient';
import { QUERY_KEYS } from '../constants/queryKeys';

export function useMovieDetails(id: number | null) {
  return useQuery({
    queryKey: QUERY_KEYS.movies.detail(id ?? 0),
    queryFn: async () => {
      const { data } = await tmdbClient.get(`/movie/${id}`);
      return data;
    },
    enabled: id !== null, // pobiera TYLKO gdy modal otwarty
  });
}
```

7. Komponenty i obsługa stanów UI

7.1. Wszystkie stany zapytania — schemat obsługi

Stan	Warunek React Query	Komponent / zachowanie
Ładowanie	isLoading === true	Wyświetl <SkeletonCard /> — siatka 12 kart z animacją shimmer
Błąd	isError === true	Wyświetl <ErrorBanner /> z komunikatem error.message + przycisk Spróbuj ponownie
Pusty wynik	data.results.length === 0	Wyświetl <EmptyState /> — ikona + tekst 'Nie znaleziono filmów'
Sukces	isSuccess === true	Renderuj siatkę <MovieCard /> z danymi
Poprzednie dane	isPlaceholderData	Przyciemnij siatkę (opacity: 0.5) — dane z poprzedniej strony

7.2. Komponent MovieCard z optimistic update

Optimistic update polega na natychmiastowym zaktualizowaniu stanu UI (dodanie do ulubionych) jeszcze przed potwierdzeniem operacji (zapis do localStorage). Użytkownik widzi zmianę bez opóźnienia.

```
// src/components/MovieCard.tsx
import { useState, useCallback } from 'react';
import { useFavorites } from '../hooks/useFavorites';
import type { Movie } from '../hooks/useFetchMovies';

const IMG_BASE = 'https://image.tmdb.org/t/p/w500';

interface Props { movie: Movie; }

export function MovieCard({ movie }: Props) {
  const { isFavorite, toggleFavorite } = useFavorites();
  const [optimisticFav, setOptimisticFav] = useState<boolean | null>(null);

  // Stan wyświetlany = optimistic (jeśli ustawiony) ?? rzeczywisty
  const displayedFav = optimisticFav ?? isFavorite(movie.id);

  const handleToggle = useCallback(async () => {
    // 1. Natychmiast zaktualizuj UI (optimistic)
    setOptimisticFav(!displayedFav);
    try {
      // 2. Wykonaj faktyczną operację
      await toggleFavorite(movie);
      // 3. Wyczyść stan optimistic — rzeczywisty stan zsynchronizowany
      setOptimisticFav(null);
    } catch {
      // 4. Rollback przy błędzie
      setOptimisticFav(null);
    }
  }, [displayedFav, toggleFavorite, movie]);

  return (
    <div className='movie-card'>
      <img
        src={movie.poster_path ? `${IMG_BASE}${movie.poster_path}` : '/no-poster.png'}
        alt={movie.title}
      />
      <h3>{movie.title}</h3>
      <p>{movie.release_date?.slice(0, 4)} • ★ {movie.vote_average.toFixed(1)}</p>
      <button
        onClick={handleToggle}
      />
    </div>
  );
}
```

```
      aria-label={displayedFav ? 'Usuń z ulubionych' : 'Dodaj do ulubionych'}
      className={`fav-btn ${displayedFav ? 'active' : ''}`}
    >
      {displayedFav ? '❤️' : '💜'}
    </button>
  </div>
);
}
```

7.3. Hook useFavorites — persystencja lokalna

```
// src/hooks/useFavorites.ts
import { useState, useCallback } from 'react';
import type { Movie } from './useFetchMovies';

const STORAGE_KEY = 'movie-browser-favorites';

function loadFavorites(): Movie[] {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY) ?? '[]');
  } catch {
    return [];
  }
}

export function useFavorites() {
  const [favorites, setFavorites] = useState<Movie[]>(loadFavorites);

  const toggleFavorite = useCallback(async (movie: Movie) => {
    setFavorites((prev) => {
      const next = prev.some((m) => m.id === movie.id)
        ? prev.filter((m) => m.id !== movie.id)
        : [...prev, movie];
      localStorage.setItem(STORAGE_KEY, JSON.stringify(next));
      return next;
    });
  }, []);

  const isFavorite = useCallback(
    (id: number) => favorites.some((m) => m.id === id),
    [favorites]
  );

  return { favorites, toggleFavorite, isFavorite };
}
```

7.4. Komponent SkeletonCard — stan ładowania

```
// src/components/SkeletonCard.tsx
export function SkeletonCard() {
  return (
    <div className='skeleton-card' aria-hidden='true'>
      <div className='skeleton-img shimmer' />
      <div className='skeleton-title shimmer' />
      <div className='skeleton-meta shimmer' />
    </div>
  );
}

// CSS shimmer animation:
// @keyframes shimmer {
//   from { background-position: -200% 0; }
//   to   { background-position: 200% 0; }
// }
// .shimmer {
//   background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
//   background-size: 200% 100%;
```

```
// animation: shimmer 1.5s infinite;  
// }
```

8. Mockowanie odpowiedzi HTTP

Mockowanie pozwala rozwijać i testować interfejs bez dostępu do prawdziwego API. Jest niezbędne przy: braku klucza API, testowaniu stanów błędów, symulacji powolnych odpowiedzi oraz testach automatycznych.

8.1. Porównanie narzędzi do mockowania

Narzędzie	Typ	Zalety	Wady / ograniczenia
MSW.js (Mock Service Worker)	Biblioteka npm	Działa w przeglądarce i Node.js (testy), interceptuje na poziomie Service Workera, nie modyfikuje kodu produkcyjnego	Wymaga konfiguracji, plik public/mockServiceWorker.js
Requestly	Rozszerzenie Chrome/Firefox	Brak kodu, redirect/modify dowolnych żądań, GUI do konfiguracji reguł, darmowy plan	Tylko przeglądarka, wymaga instalacji rozszerzenia
ModResponse	Rozszerzenie Chrome	Mock z pliku JSON, prosta konfiguracja, dobry do szybkiego prototypowania	Tylko Chrome, ograniczone opcje warunkowe

8.2. MSW.js — konfiguracja i przykładowe handlersy

```
// src/mocks/handlers.ts
import { http, HttpResponse, delay } from 'msw';

const TMDB_BASE = 'https://api.themoviedb.org/3';

export const handlers = [
  // Mock listy popularnych filmów
  http.get(`${TMDB_BASE}/movie/popular`, async ({ request }) => {
    await delay(800); // symuluj opóźnienie sieci
    const url = new URL(request.url);
    const page = Number(url.searchParams.get('page') ?? 1);
    return HttpResponse.json({
      page,
      total_pages: 10,
      results: Array.from({ length: 20 }, (_, i) => ({
        id: page * 100 + i,
        title: `Film testowy ${page}-${i + 1}`,
        overview: 'Opis testowego filmu.',
        poster_path: null,
        release_date: '2024-01-01',
        vote_average: 7.5,
        genre_ids: [28, 12],
      })),
    });
  }),

  // Mock błędu autoryzacji – do testowania ErrorBanner
  http.get(`${TMDB_BASE}/movie/popular`, () => {
    return HttpResponse.json(
      { status_message: 'Invalid API key.' },
      { status: 401 }
    );
  }),

  // Rick & Morty – mock dla testów jednostkowych
  http.get('https://rickandmortyapi.com/api/character', () => {
    return HttpResponse.json({
      info: { count: 2, pages: 1, next: null },
    });
  })
];
```

```
    results: [  
      { id: 1, name: 'Rick Sanchez', status: 'Alive', species: 'Human', image: '' },  
      { id: 2, name: 'Morty Smith', status: 'Alive', species: 'Human', image: '' },  
    ],  
  });  
}),  
];
```

```
// src/mocks/browser.ts  
import { setupWorker } from 'msw/browser';  
import { handlers } from './handlers';  
  
export const worker = setupWorker(...handlers);
```

```
// src/main.tsx — uruchom MSW tylko w trybie deweloperskim  
async function enableMocking() {  
  if (import.meta.env.DEV) {  
    const { worker } = await import('./mocks/browser');  
    return worker.start({ onUnhandledRequest: 'bypass' });  
  }  
}  
  
enableMocking().then(() => {  
  createRoot(document.getElementById('root')!).render(<App />);  
});
```

8.3. Requestly — konfiguracja bez kodu

- Zainstaluj rozszerzenie: <https://requestly.com> (Chrome, Firefox, Edge)
- Utwórz nową regułę: Modify API Response
- Warunek: URL zawiera api.themoviedb.org/3/movie/popular
- Wklej przykładowy JSON z odpowiedzią w polu Response Body
- Opcja: Add Delay → 1000 ms — symulacja wolnego internetu
- Opcja: Override Status Code → 500 — testowanie ErrorBanner

Requestly przydaje się gdy nie masz możliwości instalacji paczek npm lub chcesz szybko zmockować konkretny endpoint bez modyfikacji kodu.

9. Rozszerzenie: Infinite Scroll z useInfiniteQuery

useInfiniteQuery to specjalny hook React Query zoptymalizowany pod nieskończone ładowanie. Przechowuje wyniki wszystkich dotychczasowych stron w jednym miejscu cache.

```
// src/hooks/useInfiniteMovies.ts
import { useInfiniteQuery } from '@tanstack/react-query';
import { tmdbClient } from '../api/tmdbClient';

export function useInfiniteMovies(query = '') {
  return useInfiniteQuery({
    queryKey: ['movies', 'infinite', query],
    queryFn: async ({ pageParam = 1 }) => {
      const endpoint = query ? '/search/movie' : '/movie/popular';
      const params: Record<string, string | number> = { page: pageParam };
      if (query) params.query = query;
      const { data } = await tmdbClient.get(endpoint, { params });
      return data;
    },
    initialPageParam: 1,
    getNextPageParam: (lastPage) =>
      lastPage.page < lastPage.total_pages ? lastPage.page + 1 : undefined,
  });
}
```

```
// src/components/InfiniteMovieList.tsx
import { useEffect, useRef } from 'react';
import { useInfiniteMovies } from '../hooks/useInfiniteMovies';
import { MovieCard } from './MovieCard';
import { SkeletonCard } from './SkeletonCard';

export function InfiniteMovieList({ query = '' }) {
  const { data, fetchNextPage, hasNextPage, isFetchingNextPage } =
    useInfiniteMovies(query);
  const sentinelRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const observer = new IntersectionObserver(
      (entries) => {
        if (entries[0].isIntersecting && hasNextPage && !isFetchingNextPage) {
          fetchNextPage();
        }
      },
      { threshold: 0.1 }
    );
    if (sentinelRef.current) observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [fetchNextPage, hasNextPage, isFetchingNextPage]);

  const movies = data?.pages.flatMap((p) => p.results) ?? [];

  return (
    <>
      <div className='movie-grid'>
        {movies.map((movie) => <MovieCard key={movie.id} movie={movie} />)}
        {isFetchingNextPage && Array.from({ length: 4 }).map((_, i) =>
          <SkeletonCard key={`sk-${i}`} />
        )}
      </div>
      {/* Element obserwowany przez IntersectionObserver */}
      <div ref={sentinelRef} style={{ height: 1 }} />
    </>
  );
}
```

10. Kryteria oceny

Poniżej tabela punktowa dla zadania Movie Browser.

10.1. Punktacja szczegółowa

#	Kryterium	Wskazówka
1	Poprawna konfiguracja React Query v5 (QueryClient, QueryClientProvider, queryKeys)	Sprawdź DevTools — czy zapytania mają czytelne klucze?
2	Działający hook useFetchMovies — lista popularnych filmów	Dane ładują się przy starcie aplikacji
3	Obsługa WSZYSTKICH stanów UI: skeleton, error, empty, success	Przetestuj z wyłączoną siecią i błędnym kluczem API
4	Wyszukiwanie z debouncingiem — min. 300 ms, min. 2 znaki	Sprawdź w Network tab — czy nie ma zapytań na każdy znak?
5	Modal / strona szczegółów — useMovieDetails z lazy fetch (enabled)	Modal powinien pobierać dane DOPIERO po otwarciu
6	Ulubione z persystencją w localStorage + optimistic update	Po odświeżeniu strony ulubione muszą pozostać
7	Konfiguracja mockowania HTTP (MSW lub Requestly) — demo błędu 401	Pokaż ErrorBanner wywołany przez mock
BONUS	Infinite scroll z useInfiniteQuery + IntersectionObserver	Płynne ładowanie bez przycisku 'Następna strona'

11. Materiały i literatura

11.1. Dokumentacja techniczna

- TanStack Query v5 — <https://tanstack.com/query/latest>
- The Movie Database API — <https://developer.themoviedb.org/docs>
- Rick and Morty API — <https://rickandmortyapi.com/documentation>
- Mock Service Worker — <https://mswjs.io/docs>
- Requestly — <https://docs.requestly.com/api-client/overview>
- Vite — zmienne środowiskowe — <https://vitejs.dev/guide/env-and-mode>

11.2. Literatura przedmiotu

- Badura C., UXUI. Design Zoptymalizowany, Helion 2019
- Tidwell J., Brewer Ch., Valencia-Brooks A., Projektowanie interfejsów. Sprawdzone wzorce projektowe, Helion 2020
- Greever T., Projekt doskonały. Zadbaj o komunikację z klientem, wysoki poziom UX i zdrowy rozsądek, Helion 2022

11.3. Narzędzia wykorzystywane na zajęciach

Narzędzie	Wersja	Zastosowanie
Node.js	≥ 18.x	Środowisko uruchomieniowe
Vite	≥ 5.x	Bundler + dev server
React	≥ 18.x	Biblioteka UI
@tanstack/react-query	5.x	Zarządzanie stanem serwera
axios	≥ 1.x	Klient HTTP
msw	2.x	Mockowanie HTTP w przeglądarce
Requestly	Chrome ext	Mockowanie HTTP bez kodu
React Query DevTools	5.x	Debugowanie cache zapytań